

Defuse-Deck 3D

A turn-based card duel to defuse a bomb — procedural geometry and animation, hybrid PBR texturing, and a cinematic camera

Team LAKEBOYS

[Student 1], [Student 2], [Student 3] Matricole: [...]

Final Project Report — Interactive Graphics

Instructor: Prof. Marco Schaerf — Sapienza University of Rome — A.Y. 2025/2026

1 Introduction

Defuse-Deck 3D is a browser-based 3D puzzle/deck-builder in the style of *Balatro*, set in an underground bunker. The player is a bomb-disposal operator sitting at a workbench: by playing poker combinations with a deck of electronics-themed cards (VOLT, WIRE, CHIP, CAP suits), they charge a *defuse* meter. Every turn an enemy robot, **The Warden**, actually plays its own cards to charge the bomb's *overcharge* meter. Whoever fills their meter first wins: the player defuses the bomb, or the Warden detonates it. At every 25% of defuse progress a module of the bomb physically opens — and the camera flies in to show it happening (locks snap, an access panel slides out); if the Warden wins, the whole hierarchical model blows apart in a radial explosion.

The guiding principle of the project is that **all geometry and all animation are produced in code**: every mesh is assembled from Three.js primitives, and every motion is either an eased tween or per-frame procedural code — no 3D model or keyframe clip is ever imported. For surface detail and atmosphere the project takes a pragmatic, hybrid approach: the large environment surfaces use CC0 photographic PBR texture sets, while the gameplay-critical objects (the cards, the hazard band, the rank glyphs) keep procedurally generated textures; sound likewise mixes a handful of imported music/impact samples with live Web Audio synthesis. Every choice is explained in the sections that follow.

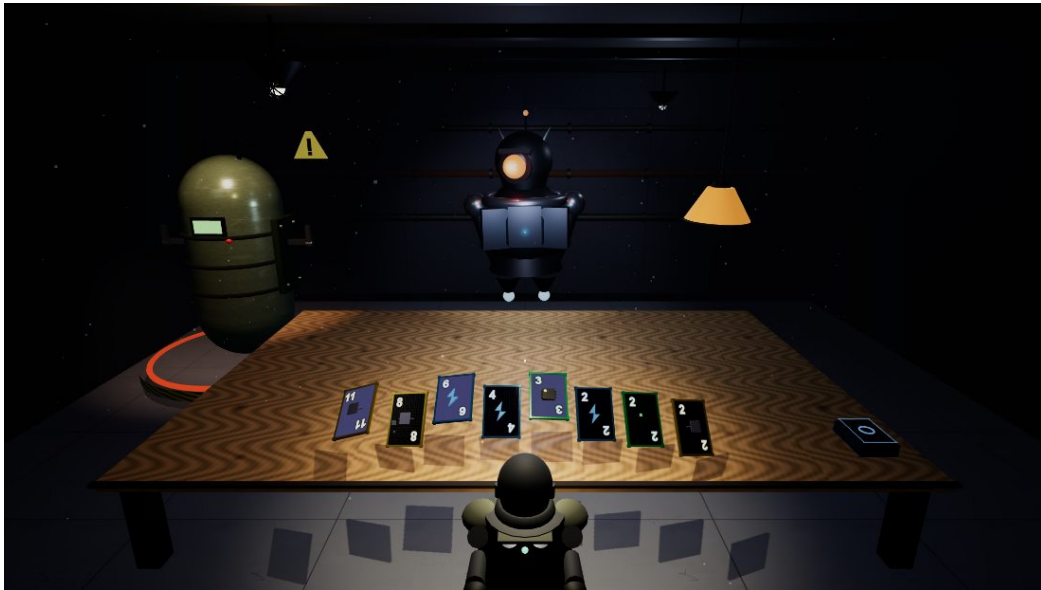


Figure 1: Third-person view: the operator's hand fanned on the workbench, the physical deck (right), the giant bomb on its hazard platform (left), the Warden hovering behind the bench, and the hanging lamp.

2 Technologies and Running the Project

The project is written in vanilla JavaScript (ES modules), with no UI framework and no mandatory build step. Two external libraries are used:

- **Three.js r167** (MIT) — renderer, scene graph, cameras, lights, shadows, materials, `Raycaster`, `OrbitControls`, `TextureLoader`, `PMREMGGenerator`;
- **tween.js 23.1.3** (MIT) — eased interpolation for all event-driven animations.

Both are resolved through an `importmap` in `index.html`, so the project runs from any static HTTP server (ES modules cannot be loaded from `file://`):

```
python3 -m http.server 3000 (from the project root, then open http://localhost:3000)
```

A Vite dev-server configuration is also included (`npm install && npm run dev`). The game targets desktop Chromium/Firefox with keyboard and mouse.

3 Software Architecture

The code base (~5,500 lines, ~25 modules) is organized in four layers with a strict dependency direction: the `core/` layer is pure logic with **zero dependencies on the DOM or Three.js**, which keeps the game rules testable and completely separate from presentation. `GameState` exposes a minimal publish/subscribe interface (`on('defuse'|'overcharge'|'turn'|'danger'|'end')`): the HUD and the 3D scene react to state events instead of being called directly.

Module	Responsibility
<code>main.js</code>	Bootstrap, wiring of all systems, <code>requestAnimationFrame</code> loop
<code>GameManager.js</code>	Turn orchestration only; delegates every domain to the modules below
<code>core/combos.js</code>	Poker-hand detection and scoring (single source of truth for both players)
<code>core/GameState.js</code>	Pure duel state, rules, pub/sub events
<code>core/EnemyAI.js</code>	The Warden's "brain": draws, evaluates and plays real cards
<code>core/jokers.js</code> , <code>difficulty.js</code>	Score modifiers and difficulty presets (pure data + functions)
<code>scene/SceneManager.js</code>	Renderer, cameras, lights, fog, environment, camera cinematics
<code>scene/*Model.js</code>	Hierarchical models: room, bomb, Warden, deck, cards, jokers
<code>scene/TextureFactory.js</code>	Texture pipeline: CC0 PBR loading + procedural card/hazard/rank maps
<code>systems/CardSystem.js</code>	3D hands: fan layout, dealing, discarding, enemy plays
<code>systems/InputManager.js</code>	Raycaster picking, hover/selection/drag, keyboard shortcuts
<code>systems/Effects.js</code> , <code>Particles.js</code>	Camera shake, screen flash, <code>THREE.Points</code> spark bursts
<code>systems/AudioManager.js</code>	Hybrid audio: mp3 samples + Web Audio synthesis
<code>systems/flight.js</code>	Shared quadratic-Bézier arc-flight helper
<code>ui/HUD.js</code>	DOM overlay: meters, score reveal, tooltips, end screen

Table 1: Source modules and their responsibilities.

4 Hierarchical Models

All models are trees of `THREE.Group` nodes built from primitives; animating a parent transforms every child, and dedicated **pivot groups** are placed exactly where a part must rotate. The most important model is the bomb:

```
Bomba (Group, uniform scale 2.8)
|-- Scocca (Group)          casing: cylinder + 2 hemispherical caps + ribs
|   '-- PannelloGroup (Group) pivot ON the surface -> linear slide-out
|   '-- Pannello + rivets (Mesh)
|-- Serratura1 / 2 (Group)   lock pivots; children: bar, clasp, head
|-- Nucleo (Group)          glowing core, timer display, wires, LED
|-- antennas (Mesh x2)      fly away in the explosion
'-- PointLight (red warning) a LIGHT as a child of the model group
```

Because each lock is a pivot *group* whose origin sits on the casing surface, opening it is a single tween of `rotation.y`; the mesh children follow. The red `PointLight` is attached **inside** the bomb group, so the light source moves and sways with the model. The 25%/50%/75% defuse thresholds re-use the same hierarchical animations: lock 1 snaps open, lock 2 snaps open, then the panel slides out safely (Figure 2b).

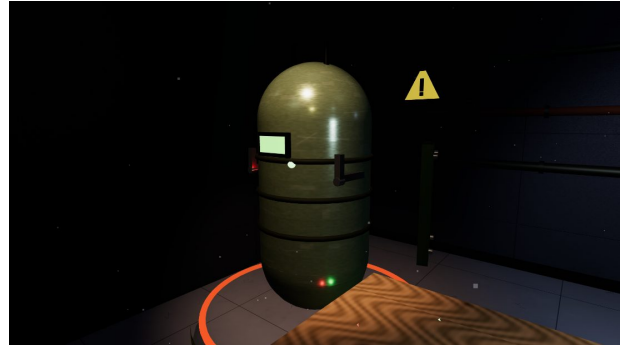
Other hierarchical models follow the same pattern:

- **Card** — Card (Group) → base (Mesh) + frame strips (×8) + Symbol (Group) + two rank labels. Each suit has its own 3D symbol: an extruded lightning bolt (`ExtrudeGeometry` from a 2D Shape), a torus connector, a microchip with gold pins, a capacitor with parallel plates.
- **The Warden and the operator** — humanoids built as multi-level chains (`Root` → `HoverPivot/Bust` → `Arm` (shoulder pivot) → `Forearm` (elbow pivot)): the deal gesture tweens shoulder and elbow pivots independently, exactly like a jointed arm.

- **Room, hanging lamp, work lights, deck, jokers** — each is a small Group hierarchy (e.g. cord→cap→shade→bulb for the lamp; rod→reflector→bulb→cage for the caged work lights; stack of slabs + discard pile for the deck; body→moving part such as a multimeter needle or rotating Tesla-coil rings for the jokers).



(a) The Warden’s turn: it “throws” its cards from the claw and they line up in a floating fan (here a Full House). Shoulder/elbow pivots drive the gesture.



(b) Bomb at 75%+ defuse: both lock pivots rotated 90°, access panel slid out on its surface pivot, LED turned green, hazard ring and metal casing visible.

Figure 2: Hierarchical models reacting to game state.

5 Textures and Audio

5.1 Texture pipeline

The scene deliberately mixes two texture sources, each where it pays off most.

Photographic PBR sets (CC0). The large, real-world surfaces — the concrete floor and walls, the wooden workbench, the painted-metal parts and the bomb platform — use free CC0 texture sets from ambientCG (*Concrete042C*, *Planks037A*, *PaintedMetal006*). Each set provides the **full PBR stack** — color (sRGB), normal (GL), roughness and, for metal, metalness — loaded through `THREE.TextureLoader` with repeat wrapping and anisotropic filtering. A single set is tiled at different densities on different surfaces (e.g. the floor at 7×6, the walls at 5×3, the platform at 2×2): `THREE.Cache` keeps one decoded image in memory while each `THREE.Texture` keeps its own repeat, so we get variety without re-fetching.

Procedural (Canvas 2D) textures. The gameplay-critical objects keep generated textures, drawn on an HTML canvas and wrapped in a `THREE.CanvasTexture`: the card faces (a dark “holographic PCB” pattern of broken bus traces, vias and an iridescent diagonal gradient, tinted with the suit’s accent color, plus a matching normal map that embosses the traces); the diagonal yellow/black **hazard band** around the bomb platform; and the **rank glyphs** (each number drawn once with a white fill, suit-colored outline and dark halo onto an unlit `MeshBasicMaterial`, so it stays readable from any angle — cached by (value, color) so identical cards share one texture). These are the elements the player reads constantly and that must stay crisp and semantically colored, which is exactly what procedural generation guarantees.

Environment reflections are procedural too: a PMREM is generated from Three.js’s code-built `RoomEnvironment` scene and applied at very low intensity (0.015), giving subtle reflections on the bomb casing and the metal card frames without washing out the dark bunker.

5.2 Audio

Audio follows the same hybrid philosophy. A small set of **imported samples** (mp3) covers the two looping music tracks (menu and match themes) and a few punchy one-shots (card draw, “mult hit” score ticks, the explosion boom and the game-over stinger). Everything else — card select/deselect, discard whoosh, deck shuffle, the ascending chord on a played hand, enemy impact, alarm, victory arpeggio — is **synthesized live** with the Web Audio API (oscillators, a reusable white-noise buffer, gain envelopes). Samples and synthesis share one `AudioContext` and one master gain, so a single mute toggle governs all sound; the context is created lazily on the first user gesture to satisfy browser autoplay policies, and music is routed through a separate gain so it sits under the SFX.

6 Rendering and Lighting

The `WebGLRenderer` uses **ACES Filmic tone mapping** (exposure 1.2), sRGB output, antialiasing, and PCF shadow maps. Exponential fog (`FogExp2`, density 0.026) gives the bunker depth without hiding the bomb or the enemy.

The lighting design combines one shadow-casting key light with cheap fills:

- a warm **SpotLight** above the bench (the only shadow caster, 1024² map) — cards and jokers cast readable shadows on the wood;
- two dramatic **spot key lights** on the bomb and on the Warden, plus a frontal **PointLight** fill for the enemy;
- the **red warning PointLight parented to the bomb**, whose intensity flickers procedurally (§8) and spikes on impacts;
- emissive materials for the lamp bulbs/shade, timer display, LED and core — glow without extra dynamic lights;
- an ambient term plus a few low-cost fills (card area, back-wall wash, top fill).

Defuse-progress indicator. Three horizontal pipes run along the back wall behind the Warden. Each has a rising activation threshold (bottom pipe first) and a distinct emissive tint (red, blue, green). As the player’s defuse meter climbs, each pipe ramps its emissive intensity with $\text{act} = \text{clamp}((\text{progress} - \text{threshold})/0.34)$ and pulses sinusoidally, and a single wall **PointLight** follows the color of the most active pipe — so the room itself visualizes how close the bomb is to being defused.

7 Camera System and Cinematics

The scene is explored with damped `OrbitControls` (distance and polar-angle limited), but on top of free navigation the project layers three **code-driven camera behaviours**, all sharing the same technique: a manual interpolation advanced by *real elapsed time* (not per-frame Δt , so it stays correct even if the browser throttles frames), with `OrbitControls` *suspended* during the move and resumed exactly at the new pose — this prevents the controls from fighting the animation or leaving the view drifted.

- **First $\rightleftharpoons$ third-person view** (key **V**) — a 750 ms `easeInOutQuad` interpolation of camera position and target between the panoramic vantage and the operator’s eyes.
- **Bomb cinematic** (`focusOnBomb`) — **the main new feature**. When a defuse module is about to open, the camera performs a three-phase *dolly-in* $\rightarrow$ *hold* $\rightarrow$ *dolly-out*: it eases in toward a three-quarter vantage on the bomb (750 ms), holds while the lock/panel animation and its particle burst play out (1500 ms), then eases back to *exactly* the previous pose (850 ms). The module opening is intentionally delayed to coincide with the camera’s arrival, so the action always happens on screen. The same cinematic, with a longer hold, frames the final detonation.
- **Bomb auto-presentation** — because the three modules face different directions, each stage tweens the bomb’s `rotation.y` so the module about to open turns to face the cinematic camera. The target yaw is derived analytically from the camera vantage ($\alpha = \text{atan2}(5.8, 6.6)$) and each module’s local surface normal.

Together, the view transition and the bomb cinematic turn state changes into directed moments instead of numbers ticking on the HUD.

8 Animation System

Beyond the camera, no keyframe clips exist in the project. Object and shader animations belong to two families that cooperate in the same frame loop.

8.1 Event-driven animations (`tween.js`)

- **Bomb defuse / failure stages** — lock pivots rotate $\pm 90^\circ$ with `Elastic.Out` (mechanical snap), the panel slides on the surface pivot, and the final **explosion** launches every child of the bomb group along its own radial trajectory with individual position/rotation tweens, while the red light spikes and the main spotlight fades to red.
- **Cards** — dealing (each card starts on the physical deck and flies into the fan with a flip, `Back.Out`, staggered); hover (lift + tilt + scale); selection (lift with a scale “punch”); a two-phase play: cards first line up facing the camera, then launch one by one toward the bomb, each ending in a spark burst.
- **Arc flights** (`flyArc` helper) — draws, discards and enemy card throws follow a quadratic Bézier

$$B(t) = (1 - t)^2 P_0 + 2(1 - t)t P_1 + t^2 P_2,$$

where P_1 is the midpoint raised above both endpoints — evaluated in code every frame inside a tween’s `onUpdate`.

8.2 Per-frame procedural animations (no keyframes)

- **Compound sine flicker** for the bomb’s red light: $f(t) = 0.55 + 0.45 |\sin(4.1t) + 0.3 \sin(13.7t)|$ — two incommensurable frequencies avoid a periodic “loop” feel. The hanging lamp sways and its bulb flickers with the same technique, and the caged work lights flicker on independent phases.
- **State-coupled animation**: the bomb LED blink rate and the core’s emissive color (incandescent red $\rightarrow$ stable green) are driven by defuse progress; the hazard ring pulses; the back-wall pipes ramp and pulse with progress (§6).
- **Idle life**: the Warden hovers and rocks, joker parts move, and 320 dust particles (`THREE.Points`) drift and slowly revolve.
- **Camera shake without drift**: shake offsets are applied to the camera *only between* `beforeRender()` and `afterRender()` and removed immediately after the draw call, so `OrbitControls` never observes the offset and the view never drifts.
- **Particle bursts**: spark positions and velocities are integrated manually each frame (random direction on a sphere, upward bias, gravity, fade), stored in a single `BufferGeometry` per burst.

9 User Interaction

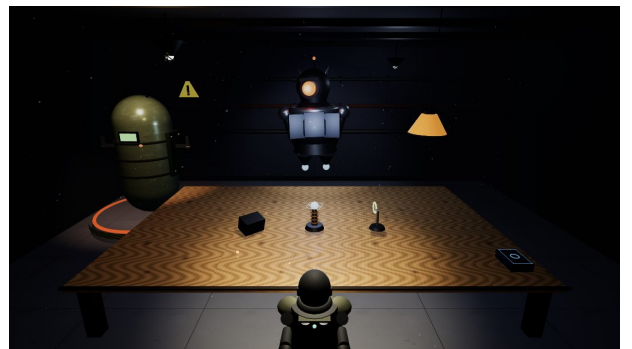
A single shared `THREE.Raycaster` is the only bridge between the mouse and the 3D scene. Mouse coordinates are converted to normalized device coordinates, a ray is cast from the camera, and the first intersected object is walked up the hierarchy until a group tagged with `userData.isCard` / `userData.isJoker` is found — so hitting any child mesh (frame strip, symbol, rank label) selects the whole card.

- **Hover** lifts a card; **click** toggles selection (max 5), with a live preview of the score the selection would produce;
- **drag on a card** rotates it freely for inspection (mouse $\Delta x \rightarrow$ yaw, $\Delta y \rightarrow$ pitch); a 4-pixel threshold distinguishes a click from a drag, and on release an `Elastic.Out` tween snaps the card back;
- **drag on the bomb** spins it around its vertical axis (X/Z locked so it stays upright), letting the player inspect all sides;
- **joker choice** at game start: the three physical objects on the bench are hoverable (highlight + tooltip) and clickable via the same raycaster;
- **keyboard**: keys **1–8** toggle cards, **Enter** plays, **X** discards, **S** re-sorts the fan, **H** suggests the best hand, **V** switches camera view, **M** mutes audio;
- during drags and camera moves `OrbitControls` is disabled, then resumed.

The hand is laid out as a fan on an arc in the XZ plane: card i gets angle $\theta_i = t_i \Theta / 2$ with $t_i \in [-1, 1]$, a position on the arc and a matching Y rotation, so every card faces the player; the total spread Θ adapts to the number of cards.



(a) First-person view (key **V**): the fan seen from the operator’s eyes; per-suit 3D symbols and procedural rank labels are readable up close.



(b) Opening phase: three joker objects (multimeter, Tesla coil, focus lens) sit on the bench and are picked with the raycaster.

Figure 3: Camera views and raycaster-driven interaction.

10 Game Logic and Enemy AI

The 40-card deck has 4 suits $\times$ values 2–11; a played hand scores, exactly like Balatro, by summing the card values into the combo’s base chips and multiplying by the combo multiplier:

$$\text{total} = (\text{baseChips}_{\text{combo}} + \sum \text{value}_{\text{cards}}) \times \text{mult}_{\text{combo}}$$

Hand	Base chips	Mult	Hand	Base chips	Mult
Straight Flush	100	$\times 8$	Three of a Kind	30	$\times 3$
Four of a Kind	60	$\times 7$	Two Pair	20	$\times 2$
Full House	40	$\times 4$	Pair	10	$\times 2$
Flush / Straight	35 / 30	$\times 4$	High Card	5	$\times 1$

Table 2: Hand-scoring table (chips $\times$ mult).

The Warden plays for real. Each enemy turn it draws 7–8 cards from its own deck and picks the best subset via `bestHand()` — a bitmask enumeration of all subsets of at most 5 cards, scored with the *same* evaluator used for the player, so the rules are provably identical. Its damage is scaled by an escalating threat multiplier $1 + \text{escalation} \cdot \text{turn}$, so the player cannot stall. The same `bestHand` routine powers the player’s “hint” button (5 per game), passing a custom scorer that includes the active joker.

Jokers are score modifiers chosen at the start (e.g. a Tesla coil that boosts VOLT cards). `applyJokers()` is a pure function returning a *new* score object — the base score is never mutated. Three difficulty presets change the detonation threshold (2000/1600/1400), the escalation rate (0.05/0.08/0.12) and the Warden’s draw size. Standard targets: defuse at 1400, detonation at 1600; 3 discards per turn; the alarm arms past 70% enemy charge.

11 Performance Choices

Choice	Rationale
Pixel ratio capped at 1.5	On Retina displays DPR 2 means $4\times$ fragments; the cap nearly halves per-frame GPU work with minimal sharpness loss.
PCFShadowMap, 1024^2 map, one shadow-casting light	Each shadow caster re-renders the scene; PCF-soft and 2048^2 were measurably slower with little visual gain in a dark scene.
Consolidated fill lights	In a forward renderer every point light costs in <i>every</i> fragment shader, even at intensity 0.
Emissive materials instead of extra lights	Lamp bulbs, LED, timer and pipes glow via emissive maps, not dynamic lights.
Texture reuse	THREE.Cache shares one decoded image across surfaces tiled at different densities; procedural rank glyphs are cached by (value, color).
Particles as THREE.Points	One geometry and one draw call per burst instead of N meshes.
Frame-time clamp ($\Delta t \leq 100$ ms)	A background-tab stall cannot teleport animations.

Table 3: Performance decisions and their rationale.

12 Course Requirements Coverage

Requirement	Where it is satisfied
Hierarchical models (nested, animated through the hierarchy)	Bomb (4 levels, multi-branch, with a child light), cards, Warden and operator arm chains, room, lamps, deck, jokers (§4); lock/panel/explosion and deal-gesture animations act on pivot groups.
No imported models or animations	Every mesh built from primitives in code; no model loaders are included; all motion is tween.js, per-frame JavaScript, or code-driven camera moves (§7, §8).
Lights and shadows	Spot key light with PCF shadow map, scenic spots, fills, ambient, red light parented to the bomb, procedural flicker (§6, §8).
Textures of several types	Full CC0 PBR sets (color/normal/roughness/metalness) for environment surfaces plus procedural color/normal maps for cards, hazard band and rank glyphs; procedural PMREM environment map (§5).
User interaction via THREE.Raycaster	Card hover/selection/drag, bomb drag-rotate and joker picking (§9).
tween.js for smooth animation	All event-driven animations, with Elastic/Back/Cubic/Quadratic easings (§8).
Camera control	Damped OrbitControls + animated first/third-person presets + bomb cinematics (§7).

Table 4: Mapping of the course requirements to the implementation.

13 User Manual

Start the game, pick a difficulty in the tutorial overlay, then click one of the three jokers on the bench. Play hands to fill the green DEFUSE bar to 1400 V before the Warden’s red OVERCHARGE bar reaches its threshold.

Input	Action
Mouse hover / click on card	Preview / toggle selection (max 5 cards)
Mouse drag on card	Rotate the card to inspect it
Mouse drag on bomb	Spin the bomb around its vertical axis
Keys 1–8	Toggle the i -th card of the fan
Enter / PLAY button	Play the selected hand (camera flies to the bomb)
X / DISCARD button	Discard selection and redraw (3 per turn)
S	Re-sort the fan
H / HINT button	Auto-select the best possible hand (5 per game)
V	Toggle first $\rightleftharpoons$ third person camera
M	Mute / unmute audio
Mouse drag / wheel (background)	Orbit / zoom the camera (OrbitControls)

Table 5: Control scheme.

14 Conclusions and Attribution

Defuse-Deck 3D integrates the topics of the course into one coherent, playable application: multi-level hierarchical modelling with pivot-based animation, a hybrid texturing pipeline (photographic PBR where realism matters, procedural where legibility matters), a designed lighting setup with real-time shadows, event-driven and per-frame animation, a directed camera system, and raycaster-driven interaction — all over a clean logic/presentation architecture. The most instructive parts were where graphics and design meet: making the bomb *physically* respond to game progress through its hierarchy, choreographing the camera to arrive exactly as a module opens, and keeping the camera shake invisible to OrbitControls.

Attribution. Libraries: Three.js r167 (MIT, <https://threejs.org>) and tween.js 23.1.3 (MIT, <https://github.com/tweenjs/tween.js>). Environment textures: CC0 PBR sets from ambientCG (<https://ambientcg.com>). Music/impact samples are used for non-commercial academic purposes. All geometry, procedural textures, procedural audio, animations and UI were written by the team.